

HoopLab

Computer Vision Basketball Shot Analysis Technical Overview & Codebase Reference

GameGala Demo · April 2026
Austin A.

Built with Flutter · YOLOv8 · Google ML Kit · On-Device Inference

\tableofcontents

1. What Is HoopLab?

HoopLab is a mobile application (iOS + Android) that uses on-device computer vision to analyze basketball shots in real time. A player records a shooting session — or uploads an existing video — and the app automatically:

- Detects every basketball shot in the footage
- Tracks the ball frame-by-frame across its entire arc
- Identifies which hoop the ball was aimed at (even when multiple hoops appear on screen)
- Determines whether the shot was a **MAKE** or **MISS** using both geometric rim-crossing analysis and the model's direct **made** detection label
- Scores shooting form across arc quality, release angle, proximity accuracy, and trajectory consistency

The result is a timestamped shot-by-shot breakdown with percentage accuracy scores and actionable form feedback — without any backend or cloud dependency at runtime.

2. Architecture

2.1 Platform Stack

Layer	Technology
Framework	Flutter 3 (Dart)
Object Detection	YOLOv8 (TFLite FP16, on-device) via ultralitics_yolo
Pose Detection	Google ML Kit Pose Detection
Video Processing	FFmpeg (ffmpeg_kit_flutter_new)
Video Playback	video_player + custom Chewie fork

The app is entirely self-contained at runtime. No server calls are made during analysis — every inference runs on the device.

2.2 Project Layout

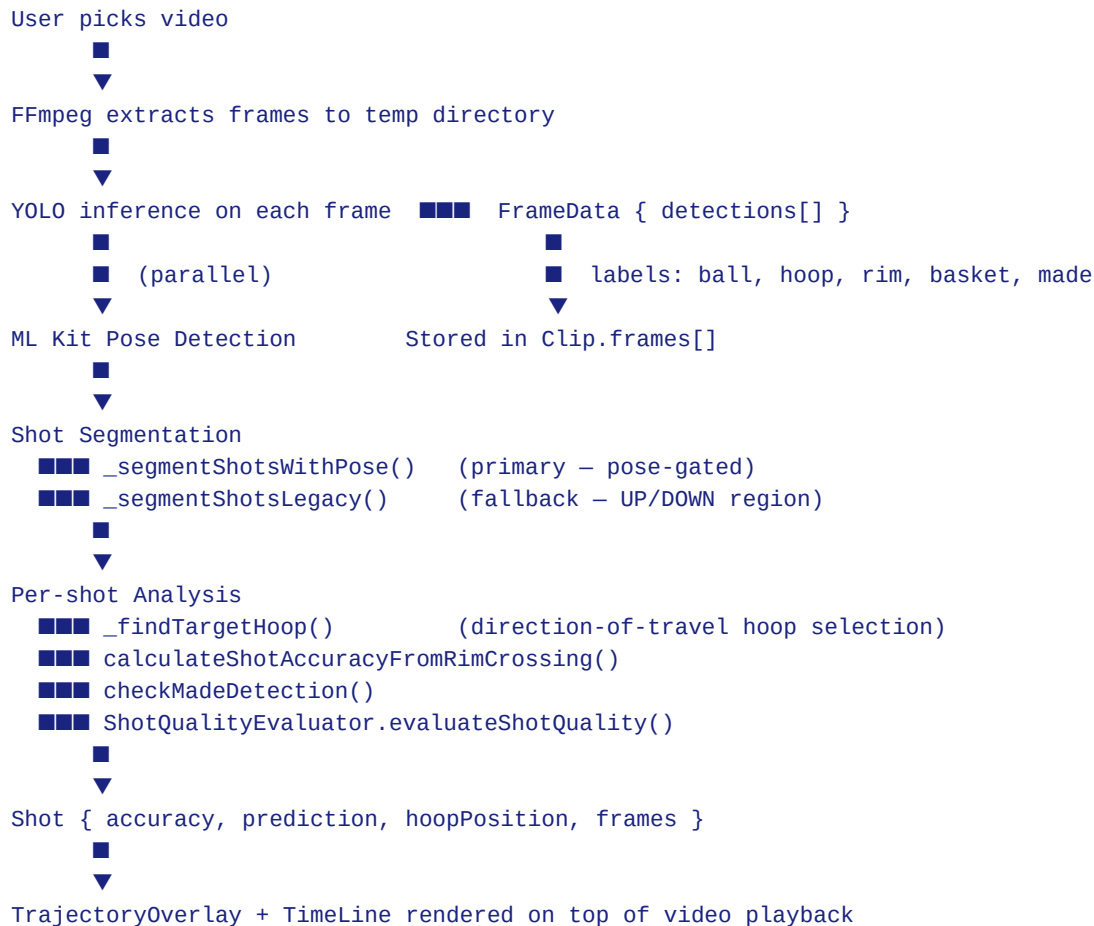
```
lib/
  ■■■ main.dart           # App entry point, Material theme
  ■■■ pages/
    ■ ■■■ method_selector.dart # Landing screen – choose input method
    ■ ■■■ camera.dart         # In-app camera recording
```

```

■   ■■■ viewer.dart           # Core analysis page (~1800 lines)
■   ■■■ live_shot_detector.dart # Real-time detection (prototype)
■   ■■■ shot_replay_viewer.dart # Replay viewer for live-detected shots
■■■ models/
■   ■■■ clip.dart             # Core data model: Clip, FrameData, Detection, Shot
■   ■■■ recorded_shot.dart    # Data model for live-detection sessions
■■■ widgets/
■   ■■■ trajectory_overlay.dart # CustomPaint: ball trajectory + pose skeleton
■   ■■■ timeline.dart         # Video scrubber with detection markers
■   ■■■ clean_video_player.dart # Video player wrapper
■■■ utils/
    ■■■ trajectory_prediction.dart # All shot math: rim crossing, made detection
    ■■■ shot_quality_evaluator.dart # Form scoring (arc, angle, consistency)
    ■■■ shooting_pose_detector.dart # ML Kit pose wrapper
    ■■■ detectionPainter.dart      # Detection bounding-box painter
    ■■■ frame_cache.dart           # Binary-search frame index cache
    ■■■ shot_analysis.dart         # Trajectory geometry helpers

```

2.3 Primary Data Flow



3. Data Models

3.1 Clip

The top-level container for a single video analysis session.

```
class Clip {
    String id, name, video_path;
    VideoInfo? videoInfo; // fps, dimensions, duration
    List<FrameData> frames; // one entry per analyzed frame
    List<Shot> shots; // detected shots (populated after segmentation)
}
```

3.2 FrameData

Represents one video frame after inference.

```
class FrameData {
    int frameNumber;
    double timestamp; // seconds
    List<Detection> detections; // all objects detected in this frame
    bool isShootingMotion; // ML Kit: someone in shooting pose?
    double shootingConfidence; // 0.0 - 1.0
    List<Pose>? poses; // raw ML Kit landmark data
}
```

3.3 Detection

A single YOLO bounding box.

```
class Detection {
    int trackId;
    BoundingBox bbox; // x1, y1, x2, y2 in pixel coordinates
    double confidence;
    String label; // "ball", "hoop", "rim", "basket", "made", ...
}
```

3.4 Shot

The result of segmenting and analyzing one shooting attempt.

```
class Shot {
    int id;
    List<FrameData> frames;
    double startTime, endTime;
    String? prediction; // "MAKE • <form feedback>" or "MISS • ..."
    double? accuracy; // 0-100 %
    Offset? hoopPosition; // target hoop center (pixel coords)
}
```

4. Analysis Pipeline

4.1 Frame Extraction

`viewer.dart` → `extractVideoFrames()`

FFmpeg extracts every frame from the video into a temporary directory at the device's native resolution. Frames are deleted immediately after analysis completes (`_framesDir.deleteSync(recursive: true)`) to avoid filling device storage.

4.2 YOLO Inference

`viewer.dart` → `analyzeVideoFrames()`

Each frame image is fed to the on-device YOLOv8 model (`best_float16.tflite`). The model returns bounding boxes with labels. The labels relevant to shot analysis are:

Label	Meaning
ball	Basketball
hoop / rim / basket	Basketball hoop (any of these)
made	Ball detected inside / passing through the hoop

Detection results are stored as `FrameData` entries in `Clip.frames`. The `made` label is produced directly by the model when it observes the ball clearly inside the hoop — this is later used as a high-confidence MAKE signal.

4.3 Pose Detection

`ShootingPoseDetector` runs Google ML Kit pose detection concurrently. It examines body landmark positions (wrists, elbows, shoulders) to determine whether a person is in a **shooting motion**. Each frame gets `isShootingMotion` and `shootingConfidence` populated.

Key heuristics used:

- Dominant wrist is above the shoulder (arm raised)
- Elbow is above shoulder height
- Body is in an upright stance (hip-to-shoulder vertical ratio)

5. Shot Segmentation

Two segmentation strategies are available. The system automatically selects pose-based when pose data is present, otherwise falls back to the legacy region-based approach.

5.1 Pose-Based Segmentation (`_segmentShotsWithPose`)

Trigger: A frame where `isShootingMotion == true` and `shootingConfidence > 0.6`.

Window: Frames are accumulated while the shooting motion is active. After the motion ends, 20 additional frames are captured to include the full ball arc and landing.

Minimum shot length: 10 frames total, ≥ 5 ball detections. Shorter sequences are discarded as false positives.

5.2 Legacy Segmentation (`_segmentShotsLegacy`)

Trigger: Ball enters the **UP region** — a box centered on the hoop, extending 4× hoop-width to each side and 2× hoop-height above.

Completion: Ball enters the **DOWN region** — below the bottom edge of the hoop.

This approach works well for backboard/front-facing camera angles but is less reliable for sideways (court-level) footage.

6. Multi-Hoop Target Selection

This is one of the most technically interesting problems HoopLab solves. Many gym recordings contain **multiple hoops** in the same frame — a foreground target hoop and one or more background hoops. Selecting the wrong hoop produces a false MISS even when the ball goes cleanly through the target.

6.1 `_findTargetHoop()` — Four-Tier Algorithm

After collecting all hoop detections across the shot's frames, unique hoops are grouped by proximity (detections within 50 px of each other are treated as the same hoop). If only one unique hoop is found, it is used directly. With multiple candidates, the following tiers run in order until one produces a confident result:

Tier 1 — Rim Crossing (most reliable)

`calculateShotAccuracyFromRimCrossing()` is run against each candidate hoop. If exactly one hoop shows a **confirmed** rim crossing (confidence: **high** or **medium**), that hoop is selected. A proximity-only estimate (no confirmed crossing) does not trigger an early return — the algorithm falls through to Tier 2.

Best for: front/backboard view where the ball's Y-coordinate clearly crosses the rim plane.

Tier 2 — Direction of Travel (primary sideways fix)

The ball always moves **toward** the target hoop. The algorithm computes the ball's overall direction vector (first detected position → last detected position), then for each hoop candidate computes the dot product:

$$\text{alignment} = (\text{travelDx}, \text{travelDy}) \cdot (\text{hoopCenter} - \text{lastBallPos})$$

The hoop with the highest positive alignment is selected. A negative alignment means the hoop is **behind** the ball's direction of travel — that hoop is skipped.

Best for: court/sideways view where the background hoop sits on the opposite side of the frame from the target.

Tier 3 — Endpoint Proximity

If the ball didn't move meaningfully (tiny magnitude travel vector), fall back to which hoop the ball's **final 30% of detections** are closest to on average.

Tier 4 — All-Frame Minimum Proximity

Last resort: which hoop had the single smallest distance to any ball detection across all frames.

6.2 Overlay Hoop Ring (trajectory_overlay.dart)

The red ring drawn on the video during playback uses the same direction-of-travel logic (implemented independently in `_getTargetHoopDetection()`). This ensures the visual indicator and the MAKE/MISS calculation always agree on which hoop is the target.

7. Shot Accuracy — Make/Miss Determination

The final MAKE/MISS decision and accuracy percentage are computed from two independent signals that are combined in priority order.

7.1 checkMadeDetection() — Primary Signal

Scans the shot's frames for any detection labeled "made" within **200 px** of the target hoop center. When found, returns:

```
accuracy = 95%
confidence = ShotConfidence.high
reason = "made label detected by model"
```

This is direct visual evidence — the model saw the ball inside the hoop. It is prioritized over all geometric methods, which is especially important in sideways view where the Y-axis crossing test is geometrically unreliable.

7.2 calculateShotAccuracyFromRimCrossing() — Fallback Signal

When no **made** label fires, a geometric rim-crossing test runs:

1. Trajectory points are interpolated (midpoints inserted) to increase resolution.
2. Scanning backward from the trajectory end, find the last point **above** the rim plane and the first point **below** it.
3. Linear interpolate between the two crossing points to find the exact X coordinate at rim height.
4. Compare this X against the rim center and width:

```
accuracy = (1 - |crossingX - rimCenterX| / rimRadius) × 100
```

This produces `ShotConfidence.high` when a complete arc (ascent + descent) is observed, `ShotConfidence.medium` for partial arcs, and falls back to proximity estimation when no crossing is found at all.

7.3 Final Result

```
final madeResult = TrajectoryPredictor.checkMadeDetection(frames, targetHoop);
final finalResult = madeResult ?? rimCrossingResult;
shot.accuracy = finalResult.accuracy;
shot.prediction = finalResult.accuracy > 50.0 ? "MAKE" : "MISS";
```

The 50% threshold means a shot must cross within one rim-radius of center to count as made — consistent with the physical margin of the actual rim.

7.4 Multi-Hoop Consistency in Rim Crossing

When `calculateShotAccuracyFromRimCrossing()` runs with `frames` provided, it dynamically updates the active hoop position from per-frame detections. In multi-hoop scenes, `_getHoopFromFrame()` uses the `initialHoopPosition` as an anchor — it always returns the hoop detection **closest to the originally selected target**, preventing camera movement or frame-to-frame ordering from drifting to the background hoop mid-calculation.

8. Shot Form Scoring

`ShotQualityEvaluator.evaluateShotQuality()` scores the shooting form independently of the MAKE/MISS result. Four components sum to 100 points:

Component	Max Points	What it measures
Arc Quality	30	Peak height ratio (1.2×–1.8× vertical hoop distance is ideal); peak position in middle third of arc
Release Angle	25	Initial trajectory angle (45°–55° is optimal for basketball)
Distance to Target	25	Closest approach of ball to hoop center
Trajectory Consistency	20	Average direction-change per step (smooth arcs score higher)

The combined score drives the form feedback string appended to the prediction: `"MAKE • Good shot form"` or `"MISS • Arc too flat or inconsistent"`.

9. Trajectory Visualization

`TrajectoryPainter` (in `trajectory_overlay.dart`) is a `CustomPainter` rendered on top of the video player. It repaints on every video position change.

Drawn elements:

Element	Visual
Ball trajectory	Orange solid line through all detected positions up to current time
Ball bounding box	Yellow rectangle at current frame
Ball position dot	Blue circle with white highlight
Target hoop ring	Red circle at target hoop center, radius = detected hoop width / 2
Predicted arc (make)	Green dashed line to hoop
Corrected arc (miss)	Green dashed ideal arc from release point to hoop center
Shot feedback text	White text with blue background near hoop

Hoop ring selection: `_getTargetHoopDetection()` computes the ball's direction of travel across all frames, then for each hoop candidate in the current frame takes the dot product against the travel vector. The hoop with the highest positive alignment is highlighted — this is consistent with the direction-of-travel Tier 2 logic in `_findTargetHoop()`.

Trajectory cleaning: Before rendering, raw detections are filtered to `confidence >= 0.5` and outliers removed by a speed check (`distance / timeDelta < 2000 px/s`, `distance < 200 px`). This suppresses stray detections without smoothing the legitimate arc.

10. Dependencies

Package	Purpose
<code>ultralytics_yolo 0.1.36</code>	On-device YOLOv8 inference via TFLite
<code>google_mlkit_pose_detection ^0.13.0</code>	Body pose landmark detection
<code>ffmpeg_kit_flutter_new ^3.2.0</code>	Frame extraction from video files
<code>video_player ^2.10.0</code>	Video playback
<code>chewie</code> (custom fork)	Video player UI controls

Package	Purpose
pro_video_editor ^0.3.0	Video trimming before analysis
camera ^0.11.3	In-app camera recording
image_picker ^1.2.0	Gallery video selection
path_provider ^2.1.5	Temp directory for extracted frames
wake_lock_plus ^1.4.0	Prevent screen sleep during analysis

11. ML Model

File: assets/best_float16.tflite

Architecture: YOLOv8 (FP16 quantized for mobile)

Size: ~5.3 MB

Classes: ball, hoop, rim, basket, made (and potentially others)

The model runs entirely on-device using the Neural Engine (iOS) or GPU delegate (Android). No network access is required during inference. The FP16 quantization halves memory bandwidth vs. FP32 with minimal accuracy loss on detection tasks.

12. Key Design Decisions

No cloud dependency at runtime. All inference is on-device. This keeps latency low, protects user video privacy, and removes the need for any server infrastructure.

Two-signal MAKE/MISS. The **made** label and rim-crossing geometry are complementary: the model label fires on clear makes in backboard view; direction-of-travel hoop selection + rim crossing handles ambiguous cases; the model label takes priority in court/sideways view where Y-axis geometry is unreliable.

Direction-of-travel hoop selection. The ball always moves toward the target hoop. Even when YOLO loses the ball mid-arc (common when it passes behind a player or the net), the overall travel direction computed from first → last detected positions reliably points to the correct hoop. This works at any camera angle.

Four-tier hoop selection with graceful degradation. Rim crossing → direction of travel → endpoint proximity → all-frame minimum. Each tier only activates if the previous tier was inconclusive. This means the most geometrically certain method wins, but the algorithm never silently fails.

Pose-gated shot segmentation. Using ML Kit shooting pose detection to gate which frames are included in a shot dramatically reduces false positives from incidental ball movement (dribbling, passes, ball rolling). Only frames where a human arm-raise is detected contribute to the shot window.

Visual/algorithmic consistency. The hoop ring drawn on the video overlay uses the same direction-of-travel logic as the MAKE/MISS calculation. The user always sees the red ring on the same hoop that determined the score.

13. Known Limitations

- **Ball tracking gaps:** YOLO occasionally loses the ball at peak arc height or when it passes behind a player. The app handles this gracefully (direction-of-travel doesn't require continuous tracking), but very sparse trajectories fall back to proximity estimation.
 - **Sideways view rim crossing:** The Y-axis rim crossing test is geometrically correct only for front-facing camera angles. In court/sideways view, the ball travels horizontally through the hoop and may not produce a Y-axis crossing. This is why **made** detection is prioritized and direction-of-travel is Tier 2 in hoop selection.
 - **Single-shot sessions:** The legacy segmentation works best for isolated shot attempts. Dense sequences (e.g., rapid-fire practice) may over-merge or under-segment shots.
 - **Pose detection accuracy:** ML Kit pose confidence varies with partial occlusion or non-standard shooting mechanics. The 0.6 confidence threshold is a balance between sensitivity and false positives.
-

HoopLab — v1.0.2 · Built with Flutter · Powered by YOLOv8 + Google ML Kit